PROJET OLYMPUS — Cahier des Charges · Module BIZON

ECE Paris · Prépa Intégrée 2025
Confidentiel — Usage interne · v1.0 — 14/04/2026

Table des Matières

1. Présentation Générale du Projet
   - 1.1 Vision & Objectifs
   - 1.2 Contexte & Motivation
2. Périmètre Fonctionnel
   - 2.1 Backlog Produit — 15 User Stories
   - 2.2 Fonctionnalités détaillées par module
3. Architecture Technique
   - 3.1 Stack technologique
   - 3.2 Diagramme des composants & flux
   - 3.3 Structure du repository
4. Planning & Sprints Agiles
   - 4.1 Sprint 1 — Infrastructure & connexion API
   - 4.2 Sprint 2 — Moteur mathématique & stratégies
   - 4.3 Sprint 3 — Interface web & visualisations
   - 4.4 Sprint 4 — Finitions, tests & livrables
   - 4.5 Gantt Chart
5. Contraintes & Exigences Non-Fonctionnelles
6. Risques & Mitigations
7. Métriques de Succès & KPIs
8. Livrables Obligatoires
9. Roadmap Post-MVP
10. Checklist Setup Day 1

1. Présentation Générale du Projet

1.1 Vision & Objectifs

Le module **Bizon** s'inscrit dans le cadre du **Projet Olympus** en tant que second composant majeur, aux côtés du modèle vocal Olympe. Bizon est un **moteur de trading algorithmique intelligent**, fonctionnant entièrement en local, conçu pour analyser les marchés financiers et exécuter automatiquement des ordres d'achat/vente sur des périodes courtes et modulables (scalping, day trading, swing trading).

Objectif	Valeur cible	Méthode de vérification
Précision des signaux (backtesting)	≥ 60% win rate sur 100 trades	Script backtest automatisé
Latence décision → ordre (mode réel)	< 500 ms	time.perf_counter()
Coût total du projet	0 € (open-source)	Stack open-source uniquement
Lignes de code Python	≥ 200 lignes	`wc -l` sur fichiers `.py`
Durée de développement	4 semaines (sprints)	Kanban / GitHub Projects
Support multi-actifs	≥ 5 instruments	Actions, crypto, forex, indices

1.2 Contexte & Motivation

Étudiant en prépa intégrée à l'ECE Paris, passionné d'intelligence artificielle et de mathématiques appliquées, ce projet prolonge le Projet Olympus vers le domaine de la finance quantitative. Bizon vise à démontrer qu'un modèle de trading algorithmique avancé peut être construit avec des outils 100% gratuits et locaux, sans abonnement à des plateformes propriétaires.

Porteur	Étudiant ECE Paris — Prépa Intégrée 2e année
Plateforme	Machine locale (Windows / macOS / Linux) — auto-détection hardware
Durée	4 semaines de développement actif · 10–15h/semaine
Budget	0 € — Stack entièrement open-source
API Trading	Alpaca Markets (paper trading gratuit + réel) ou Binance Testnet

2. Périmètre Fonctionnel

2.1 Backlog Produit — 15 User Stories Prioritaires

#	Priorité	User Story	Critère d'acceptation	Pts
US-01	MUST	Je lance Bizon et il détecte automatiquement mon hardware (CPU/GPU/RAM) et adapte ses paramètres	Détection correcte, rapport affiché dans l'interface	5

#	Priorité	User Story	Critère d'acceptation	Pts
US-02	MUST	Je configure une stratégie en mode Test (paper trading) et Bizon achète/vend sans argent réel	Ordres simulés, P&L fictif affiché en temps réel	8
US-03	MUST	Je passe en mode Réel avec confirmation explicite et Bizon exécute des vrais ordres via API	Ordres réels exécutés, confirmation avant premier trade	8
US-04	MUST	Je visualise le graphique en chandelier (OHLCV) de l'actif sélectionné dans l'interface web	Graphique interactif avec zoom, en temps quasi réel	5
US-05	MUST	Le modèle calcule les indicateurs techniques (RSI, MACD, Bollinger Bands, EMA) automatiquement	Indicateurs affichés sur le graphique, recalcul à chaque bougie	5
US-06	MUST	Je définis la période de trading (1min, 5min, 15min, 1h, 4h) et Bizon s'adapte	Toutes les périodes disponibles, stratégie recalibrée	5
US-07	SHOULD	Je vois le P&L (Profit & Loss) total, le taux de victoire et le drawdown maximum en direct	Dashboard KPIs mis à jour toutes les 5 secondes	5
US-08	SHOULD	Bizon applique automatiquement un stop-loss et un take-profit sur chaque ordre	Paramètres configurables, exécution automatique garantie	8
US-09	SHOULD	Je lance un backtest sur des données historiques et j'obtiens un rapport de performance	Rapport PDF généré avec métriques, courbe equity	8
US-10	SHOULD	Je sélectionne l'actif à trader (crypto, actions, forex) depuis un menu déroulant	Liste d'actifs configurables, switch sans redémarrage	3
US-11	COULD	Bizon affiche des alertes visuelles et sonores lors d'un signal d'achat ou de vente	Notification navigateur + son optionnel	3
US-12	COULD	Je consulte l'historique complet des trades avec horodatage, prix et résultat	Tableau paginé, export CSV possible	5
US-13	COULD	Le modèle intègre un filtre de sentiment de marché basé sur des données macro	Score sentiment 0-100, influence pondérée sur les signaux	8
US-14	COULD	Le repo GitHub est documenté avec README + GIF démo	README complet, GIF démo, badge version	3
US-15	COULD	Une stratégie ML (Random Forest / LSTM) remplace/complète les indicateurs classiques	Modèle entraîné sur données historiques, précision > 60%	13

Légende : MUST HAVE — obligatoire MVP · SHOULD HAVE — important · COULD HAVE — bonus

2.2 Fonctionnalités Détaillées par Module

Module Détection Hardware [system_info]

Attribut	Valeur
Technologie	`psutil` + `platform` + `torch.cuda.is_available()`
Détection	CPU (cœurs, fréquence), RAM disponible, GPU (CUDA/MPS/CPU)
Adaptation	Fréquence de calcul, taille des batches de données, parallélisme
Rapport	Affiché au démarrage dans l'interface web

Module Données Marché [data_feed]

Attribut	Valeur
Technologie	`yfinance` (gratuit, sans clé) + `ccxt` (crypto) + WebSocket broker
Données	OHLCV (Open, High, Low, Close, Volume) en temps quasi-réel
Périodes	1m, 5m, 15m, 1h, 4h, 1j (modulables)
Cache local	SQLite ou CSV compressé pour backtesting & offline
Latence	< 2 secondes pour données live

Module Moteur Mathématique [engine]

Attribut	Valeur
Technologie	`pandas-ta` + `ta-lib` (indicateurs techniques)
Indicateurs	RSI, MACD, Bollinger Bands, EMA 9/21/50, ATR, Volume OBV
Stratégies	Croisement EMA, RSI overbought/oversold, MACD divergence, breakout BB
Génération signaux	`BUY`, `SELL`, `HOLD` avec score de confiance 0–100%
Risk management	Stop-loss automatique (ATR × multiplicateur), take-profit configurable

Module API Trading [broker]

Attribut	Valeur
Broker papier (Test)	Alpaca Paper Trading (gratuit, sans depot) ou Binance Testnet
Broker réel	Alpaca Live ou Binance Spot (clés API dans `.env`)
Mode Test	Ordres fictifs, solde virtuel configurable (ex : 10 000 €)
Mode Réel	Ordres réels, confirmation double avant activation
Sécurité	Clés API jamais dans le code — uniquement dans `.env` (non commité)

Module Interface Web [dashboard]

Attribut	Valeur
Technologie	`Gradio 4.x` ou `Streamlit` (web local, `http://localhost:8501`)
Fonctionnalités	Graphique OHLCV interactif, KPIs P&L, historique trades, config stratégie

Attribut	Valeur
Mode toggle	Bouton Test / Réel avec confirmation modale
Accès	Navigateur uniquement (pas de déploiement cloud)

3. Architecture Technique

3.1 Stack Technologique

Couche	Technologie	Version	Rôle	Licence
Langage	Python	3.10+	Orchestration générale	PSF
Données marché	yfinance / ccxt	latest	OHLCV temps réel + historique	MIT / MIT
Indicateurs	pandas-ta	0.3.x	Calcul indicateurs techniques	MIT
Machine Learning	scikit-learn / PyTorch	1.x / 2.x	Stratégies ML (bonus)	BSD / BSD
Broker API	Alpaca / Binance	REST + WebSocket	Passage d'ordres test & réel	Alpaca ToS
Interface	Gradio 4.x ou Streamlit	4.x / 1.x	Dashboard web local	Apache 2.0
Base de données	SQLite	natif Python	Cache données & historique	Domaine public
Visualisation	Plotly	5.x	Graphiques financiers interactifs	MIT
Versioning	Git + GitHub	—	Code & CI/CD	GPL

3.2 Flux de Données — Pipeline Principal

Le pipeline Bizon suit une architecture modulaire en boucle continue :

Étape	Composant	Input	Output	Latence
1	DataFeed	Symbole + période	OHLCV DataFrame	< 2s
2	Engine	DataFrame OHLCV	Indicateurs calculés	< 100ms
3	Strategy	Indicateurs	Signal BUY/SELL/HOLD + score	< 50ms
4	RiskManager	Signal + solde	Ordre avec SL/TP calculés	< 10ms
5	Broker (mode Test)	Ordre	Confirmation fictive + P&L	< 50ms
5	Broker (mode Réel)	Ordre	Exécution réelle via API	< 500ms
6	Dashboard	Tous les états	Mise à jour interface web	< 1s

Étape	Composant	Input	Output	Latence
TOTAL	—	Signal brut marché	Ordre exécuté + interface à jour	< 4s

3.3 Structure du Repository GitHub

```
projet-bizon/
├── src/
│   ├── main.py                # Boucle principale & orchestration
│   ├── hardware/
│   │   └── detector.py        # Détection CPU/GPU/RAM + adaptation
│   ├── data/
│   │   ├── feed.py            # yfinance + ccxt : OHLCV temps réel
│   │   └── cache.py           # SQLite : stockage données historiques
│   ├── engine/
│   │   ├── indicators.py      # pandas-ta : RSI, MACD, BB, EMA, ATR
│   │   ├── strategy.py        # Logique signaux BUY/SELL/HOLD
│   │   └── risk_manager.py    # Stop-loss, take-profit, position sizing
│   ├── broker/
│   │   ├── paper_broker.py    # Mode Test — ordres fictifs + P&L simulé
│   │   ├── real_broker.py     # Mode Réel — Alpaca / Binance API
│   │   └── order_manager.py   # Gestion & suivi des ordres ouverts
│   ├── ml/                    # (BONUS)
│   │   ├── trainer.py         # Entraînement Random Forest / LSTM
│   │   └── predictor.py       # Inférence locale du modèle ML
│   ├── interface/
│   │   └── app.py             # Gradio/Streamlit : dashboard web
│   └── utils/
│       ├── error_handler.py   # Gestion erreurs & fallbacks
│       └── logger.py          # Logging structuré (fichier + console)
├── tests/
│   ├── backtest.py            # Backtest sur données historiques
│   └── test_engine.py         # Tests unitaires moteur mathématique
├── data/
│   └── cache/                 # Données OHLCV en cache local
├── docs/
│   ├── architecture.png       # Diagramme des composants
│   └── rapport.pdf            # Rapport technique
├── README.md                  # Setup 5min + badges + GIF démo
├── requirements.txt
├── Makefile                   # make run / make test / make backtest
└── .env.example               # Template clés API (jamais commité)
```

4. Planning & Sprints Agiles

Sprint 1 — Infrastructure & Connexion API (Semaine 1)

Objectif : Bizon se connecte aux données marché et passe ses premiers ordres fictifs · Budget : 12h

Tâche	Description	Durée	Fichier cible
T1.1	Détection hardware : psutil + rapport JSON	1h	`src/hardware/detector.py`

Tâche	Description	Durée	Fichier cible
T1.2	Module DataFeed : yfinance OHLCV + cache SQLite	3h	`src/data/feed.py`
T1.3	Connexion API Alpaca Paper Trading (test)	2h	`src/broker/paper_broker.py`
T1.4	Module ordre fictif : BUY/SELL + suivi P&L	2h	`src/broker/order_manager.py`
T1.5	Tests : récupération 100 bougies, premier ordre fictif	2h	`tests/test_engine.py`
T1.6	Commit GitHub + README v0.1 + `.env.example`	1h	`README.md`

Sprint 2 — Moteur Mathématique & Stratégies (Semaine 2)

Objectif : Bizon calcule les indicateurs et génère ses premiers signaux · Budget : 13h

Tâche	Description	Durée	Fichier cible
T2.1	Calcul RSI, MACD, Bollinger Bands, EMA 9/21	3h	`src/engine/indicators.py`
T2.2	Logique stratégie : croisement EMA + RSI confirmation	3h	`src/engine/strategy.py`
T2.3	Risk manager : ATR-based stop-loss + take-profit auto	2h	`src/engine/risk_manager.py`
T2.4	Intégration pipeline : DataFeed → Engine → Broker	2h	`src/main.py`
T2.5	Tests : 50 signaux générés, vérification logique	2h	`tests/test_engine.py`
T2.6	Démo interne : 1h de paper trading automatisé	1h	—

Sprint 3 — Interface Web & Visualisations (Semaine 3)

Objectif : Dashboard complet, mode Test/Réel fonctionnel, backtest opérationnel · Budget : 13h

Tâche	Description	Durée	Fichier cible
T3.1	Interface Gradio v1 : graphique OHLCV Plotly + KPIs	4h	`src/interface/app.py`
T3.2	Toggle Test / Réel avec confirmation modale	2h	`src/interface/app.py`
T3.3	Historique trades : tableau paginé + export CSV	2h	`src/interface/app.py`
T3.4	Module backtest sur données historiques (N jours)	2h	`tests/backtest.py`
T3.5	Gestion erreurs & reconnexion automatique API	1h	`src/utils/error_handler.py`
T3.6	Logging structuré (fichier + console, niveaux)	1h	`src/utils/logger.py`
T3.7	README v1 complet + GIF démo enregistré	1h	`README.md`

Sprint 4 — Finitions, Sécurité & Livrables (Semaine 4)

Objectif : Code production-ready, documentation complète, vidéo démo · Budget : 10h

Tâche	Description	Durée	Fichier cible
T4.1	Audit sécurité : clés API, .gitignore, .env strict	1h	`.env.example`, `.gitignore`
T4.2	README final : badges, GIF démo, setup 5min	2h	`README.md`
T4.3	Rapport PDF 2 pages : archi, métriques, challenges	2h	`docs/rapport.pdf`
T4.4	Code review : modules propres, docstrings, types	1h	Tous les `.py`
T4.5	`requirements.txt` + `Makefile` + `.env.example` finaux	0.5h	Racine projet
T4.6	Tag GitHub v1.0.0 + Release Notes	0.5h	GitHub Release
T4.7	(BONUS) Modèle ML Random Forest sur données historiques	3h	`src/ml/trainer.py`

4.5 Gantt Chart — Timeline 4 Semaines

Module / Tâche	S1	S2	S3	S4
Setup environnement & hardware detector	▇▇▇			
DataFeed (yfinance + cache SQLite)	▇▇▇	▇		
Broker Paper Trading (mode Test)	▇▇▇	▇		
Moteur indicateurs (RSI, MACD, BB, EMA)		▇▇▇		
Logique stratégie & génération signaux		▇▇▇		
Risk Manager (SL/TP automatique)		▇▇		
Mode Réel (Alpaca/Binance Live)		▇	▇▇	
Interface web (Gradio + Plotly)			▇▇▇	▇
Backtest historique + rapport			▇▇▇	
Gestion erreurs & logging			▇▇	
README + Documentation	▇		▇	▇▇▇
(BONUS) Modèle ML LSTM/RF				▇▇▇

▇▇▇ *= Travail actif* · ▇ *= Finalisation & tests*

5. Contraintes & Exigences Non-Fonctionnelles

Performance

- Latence totale signal → ordre (mode Réel) : < 500 ms
- Latence recalcul indicateurs : < 100 ms par bougie
- Utilisation RAM maximale : < 2 Go (sans modèle ML) / < 4 Go (avec ML)
- Fréquence de mise à jour du dashboard : toutes les 5 secondes

Sécurité

- Clés API (Alpaca, Binance) **jamais** commitées dans Git — fichier `.env` uniquement
- Mode Réel protégé par confirmation double dans l'interface (bouton + saisie du mot `CONFIRMER`)
- Stop-loss obligatoire sur tout ordre en mode Réel (protection capital)
- Limite de perte journalière configurable (circuit breaker automatique)

Disponibilité

- Fonctionnement 100% local (aucun cloud requis en mode Test)
- Reconnexion automatique à l'API broker en cas de déconnexion
- Reprise de session sans perte de l'état des trades ouverts
- Compatible Windows 10+, macOS 12+, Ubuntu 20.04+

Portabilité

- Compatible CPU-only (pas de GPU obligatoire) — adaptation automatique
- Installation reproductible en < 15 minutes via `make install`
- Support multi-OS grâce à `pathlib` et chemins relatifs

Qualité code

- Modules indépendants avec interfaces Python typées (`typing`)
- Docstrings sur toutes les fonctions publiques
- Coverage tests ≥ 60% sur modules critiques (engine, broker)
- Zéro secret/token commité dans Git (`.env` + `.gitignore` stricts)

6. Risques & Mitigations

#	Risque	Prob.	Impact	Mitigation
R1	Clés API exposées dans Git	Haute	Critique	`.gitignore` strict dès le Day 1, `pre-commit` hook de vérification
R2	Activation accidentelle du mode Réel	Moyenne	Critique	Double confirmation + variable d'env `BIZON_MODE=paper` par défaut

#	Risque	Prob.	Impact	Mitigation
R3	Perte financière incontrôlée (mode Réel)	Moyenne	Fort	Circuit breaker : stop automatique si drawdown > X% configurable
R4	API Alpaca/Binance indisponible	Faible	Fort	Fallback mode paper local, cache données last known, alerte dashboard
R5	Données yfinance incorrectes ou manquantes	Moyenne	Moyen	Validation schema DataFrame, fallback source alternative (ccxt)
R6	Stratégie non profitable (faux signaux)	Haute	Moyen	Mode Test obligatoire avant mode Réel, backtest min. 30 jours requis
R7	Surcharge CPU sur machine peu puissante	Moyenne	Moyen	Détection hardware → adaptation fréquence calcul + mode éco
R8	Manque de temps (examens ECE)	Haute	Fort	Prioriser US-01 à US-10 · Sacrifier ML et alertes sonores

7. Métriques de Succès & KPIs

7.1 KPIs Techniques

Métrique	Cible MVP	Méthode de mesure	Statut
Win rate (backtest 30 jours)	≥55%	`tests/backtest.py` automatisé	⬜ À mesurer
Latence signal → ordre (mode Test)	<500 ms	`time.perf_counter()` pipeline	⬜ À mesurer
Latence calcul indicateurs	<100 ms	Mesure isolée module engine	⬜ À mesurer
Drawdown maximum (backtest)	<15%	Courbe equity backtest	⬜ À mesurer
Taux d'erreur API broker	<5%	Compteur dans `order_manager.py`	⬜ À mesurer
Lignes de code Python	≥200 lignes	`find . -name '*.py' | xargs wc -l`	⬜ À mesurer
Modules Python distincts	≥6 modules	Structure `src/` répertoire	⬜ À mesurer

7.2 KPIs Portfolio & CV

Métrique	Cible	Vérification
Stars GitHub	≥5	GitHub Insights
Vues README (30 jours)	≥50	GitHub Traffic
Commits Git	≥25	`git log --oneline | wc -l`

Métrique	Cible	Vérification
README avec GIF démo	Oui	Visuel dans `README.md`
Rapport PDF 2 pages	Oui	`docs/rapport.pdf` livré
Aucune clé API exposée	Oui	Scan `git-secrets` ou `trufflehog`

8. Livrables Obligatoires

L1 — Repository GitHub Public (MUST)

- Code source complet modulaire (`src/`, `tests/`, `data/`, `docs/`)
- `README.md` avec GIF démo animé (dashboard en action)
- Setup reproductible en < 10 minutes (`make install` + `make run`)
- Tag release `v1.0.0` avec Release Notes
- Badge Python, badge License, badge `Mode: Paper Trading`
- `.env.example` documenté (aucune vraie clé)

L2 — Rapport Technique PDF 2 pages (MUST)

- Architecture : diagramme des composants + flux de données
- Métriques : win rate, latences mesurées, drawdown max
- Challenges & solutions : gestion du risque, qualité des données, faux signaux
- Résultats backtest : courbe equity sur 30 jours minimum

L3 — Démo Backtest (MUST)

- Backtest sur un actif réel (ex : BTC/USDT ou AAPL) sur 30 jours
- Rapport généré automatiquement : win rate, P&L total, nb trades, drawdown
- Fichier CSV exporté avec historique des trades

L4 — Modèle ML (COULD — Bonus)

- Entraînement Random Forest ou LSTM sur données historiques locales
- Intégration comme stratégie alternative dans le pipeline
- Précision > 60% en validation croisée

9. Roadmap Post-MVP — Bizon v2

Phase 2 — Stratégies Avancées (Semaines 5–6)

- Intégration LSTM pour prédiction de prix à court terme
- Stratégie multi-actifs simultanés (portfolio dynamique)
- Détection de patterns chartistes (tête-épaules, double bottom)

- `pip install torch transformers`

Phase 3 — Intégration Olympe (Semaines 7–8)

- Connexion inter-modules Olympus : commandes vocales pour Bizon
- *"Olympe, lance Bizon sur BTC en mode test 15 minutes"*
- Rapports vocaux : *"Olympe, quel est le P&L de Bizon aujourd'hui ?"*
- Interface unifiée Olympus (vocal + dashboard trading)

Phase 4 — Scale & Production (Mois 2+)

- Déploiement sur Raspberry Pi 4 ou serveur home (24h/24)
- API REST Flask → application mobile pour monitoring
- Connexion à Interactive Brokers pour marchés européens réglementés
- Fine-tuning stratégies sur données personnalisées + reporting hebdomadaire automatique

10. Checklist Setup Day 1

Exécuter ces commandes dans l'ordre pour avoir un environnement 100% fonctionnel en ~30 minutes :

#	Étape	Commande / Action
1	Cloner le repo	`git clone https://github.com/TON_USERNAME/projet-bizon.git && cd projet-bizon`
2	Créer l'environnement Python	`python3 -m venv bizon-env && source bizon-env/bin/activate`
3	Installer les dépendances	`pip install -r requirements.txt`
4	Configurer les variables d'env	`cp .env.example .env` puis éditer `.env` (mode paper par défaut)
5	Tester la récupération données	`python -c "import yfinance as yf; print(yf.download('BTC-USD', period='1d'))"`
6	Créer compte Alpaca Paper	Inscription sur `alpaca.markets` → Paper Trading → copier les clés dans `.env`
7	Tester connexion broker	`make test-broker` → doit afficher le solde fictif ($100 000 par défaut)
8	Lancer le détecteur hardware	`python src/hardware/detector.py` → rapport CPU/GPU/RAM
9	Premier lancement complet	`make run` → dashboard ouvert sur `http://localhost:8501`
10	Premier paper trade manuel	Sélectionner BTC/USDT, période 5min, cliquer **Start Bizon (Test)**

Projet Olympus — Module Bizon · Cahier des Charges v1.0
ECE Paris · Prépa Intégrée · 2025–2026
Stack : Python · pandas-ta · yfinance · Alpaca API · Gradio · Plotly · SQLite